

LF11c:90:Klausurersatzleistung:Konverterbau

Lösungen

Karsten Reincke

GS-LDK

7. September 2026

*Diese Präsentation stammt aus dem OER-Projekt **proTirone**. Es wurde auf Basis von **proScientia.ltx** entwickelt und ist **CC-BY-4.0** lizenziert. Gemäß CC-BY-4.0/§5 werden proTirone-Materialien so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung.*

initial in einer Zeile erhalten:

```
1, "2026-01-01", 30, 32, 20, 18, 15, 15, 2, "2026-01-02", 30, 35, 25, 22, 20, 18, 3, "2026-01-03", 35, 38, 20, 18, 15, 15, 4, "2026-0
```

manuell in CSV umgewandelt = Zeilenweise formatiert, Komma am Zeilenende entfernt

```
1 id, exercise-date, 3 * (targetvalue, is-value)
```

```
1, "2026-01-01", 30, 32, 20, 18, 15, 15
2, "2026-01-02", 30, 35, 25, 22, 20, 18
3, "2026-01-03", 35, 38, 20, 18, 15, 15
4, "2026-01-04", 35, 37, 30, 28, 25, 23
5, "2026-01-05", 40, 38, 30, 28, 25, 23
6, "2026-01-06", 0, 38, 0, 22, 0, 14
7, "20260107", 30, 44, 35, 33, 40, 29
8, "2026-01-08", 45, 48, 35, 33, 30, 28
9, "2026-01-09", 45, 49, 40, 39, 35, 32
10, "2026-01-10", 50, 54, 40, 38, 35, 33
11, "2026-01-11", 50, 54, 45, 42, 40, 38
12, "2026-01-12", 55, 57, 45, 44, 40, 36
13, "2026-01-13", 0, 0, 0, 0, 0, 0
14, "15.1.2026", 60, 67, 50, 46, 45, 42
15, "2026-01-14", 55, 58, 50, 48, 45, 43
```

LF11c::EL:Konverter:xyz→csv:Step-03

Datum in Datensatz 7 + 14 bereinigt, Datensätze 14/15 vertauscht

```
1, "2026-01-01", 30, 32, 20, 18, 15, 15
2, "2026-01-02", 30, 35, 25, 22, 20, 18
3, "2026-01-03", 35, 38, 20, 18, 15, 15
4, "2026-01-04", 35, 37, 30, 28, 25, 23
5, "2026-01-05", 40, 38, 30, 28, 25, 23
6, "2026-01-06", 0, 38, 0, 22, 0, 14
7, "2026-01-07", 30, 44, 35, 33, 40, 29
8, "2026-01-08", 45, 48, 35, 33, 30, 28
9, "2026-01-09", 45, 49, 40, 39, 35, 32
10, "2026-01-10", 50, 54, 40, 38, 35, 33
11, "2026-01-11", 50, 54, 45, 42, 40, 38
12, "2026-01-12", 55, 57, 45, 44, 40, 36
13, "2026-01-13", 0, 0, 0, 0, 0, 0
14, "2026-01-14", 55, 58, 50, 48, 45, 43
15, "2026-01-15", 60, 67, 50, 46, 45, 42
```

Testdaten abgespalten:

```
1, "2026-01-01", 30, 32, 20, 18, 15, 15
2, "2026-01-02", 30, 35, 25, 22, 20, 18
3, "2026-01-03", 35, 38, 20, 18, 15, 15
```



LF11c::EL:Konverter:xyz→json:Step-04:MVP

```
1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # for converting a csv file of planking data into a json file: MVP
3
4  # intermediate representation and hard-coded test data set of planking data
5  td_hard_coded=[
6      { "set": 1, "day": "2026-01-01",
7        "shall1": 30, "is1": 32, "shall2": 20, "is2": 18, "shall3": 15, "is3": 15},
8      { "set": 2, "day": "2026-01-02",
9        "shall1": 30, "is1": 35, "shall2": 25, "is2": 22, "shall3": 20, "is3": 18} ]
10
11 # adapters to read planking-data from a file:
12 def fill_from_csv(filename) :
13     print(f"reading from csv file <{filename}>")
14     planking_data=[]
15     return planking_data
16
17 # adapters to write planking-data into a file
18 def write_as_csv(filename,planking_data):
19     print(f"writing into csv file <{filename}>")
20
21 # main
22 planking_data_as_csv="res.lf/planking_data.csv"
23 planking_data=td_hard_coded
24 if (planking_data):
25     write_as_csv(planking_data_as_csv,planking_data)
26
```

LF11c::EL:Konverter:xyz→json:Step-04:VP-0.1

```

1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # for converting a csv file of plankting data into a json file: VP-0.1
3
4  # intermediate representation and hard-coded test data set of plankting data
5  td_hard_coded=[
6      { "set": 1, "day": "2026-01-01",
7        "shall1": 30, "is1": 32, "shall2": 20, "is2": 18, "shall3": 15, "is3": 15},
8      { "set": 2, "day": "2026-01-02",
9        "shall1": 30, "is1": 35, "shall2": 25, "is2": 22, "shall3": 20, "is3": 18} ]
10
11 # adapters to read plankting-data from a file:
12 def fill_from_csv(filename) :
13     print(f"reading from csv file <{filename}>")
14     plankning_data=[]
15     return plankning_data
16
17 # adapters to write plankting-data into a file
18 def write_as_csv(filename,plankning_data):
19     print(f"writing into csv file <{filename}>")
20     ouf=open(filename,"w")
21     # keep it simple: let the program crash if filename does not exist
22     for ds in plankning_data:
23         ouf.write(f"{ds["set"]},{ds["day"]},{ds["shall1"]},{ds["is1"]},"
24                 f"{ds["shall2"]},{ds["is2"]},{ds["shall3"]},{ds["is3"]}\n")
25     return ouf.close()
26
27 # main
28 plankning_data_as_csv="res.lf/plankning_data.csv"
29 plankning_data=td_hard_coded
30 if (plankning_data):
31     write_as_csv(plankning_data_as_csv,plankning_data)
32

```

LF11c::EL:Konverter:xyz→json:Step-05:VP-0.2a

```

1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # functions for reading and converting a csv file of plankting data: VP-1.0
3
4  # (1) adapter to read plankting data from a csvfile
5  def fill_from_csv(filename) :
6      print(f"reading from csv file <{filename}>")
7      plankting_data=[]
8      inf=open(filename,"r")
9      # keep it simple: let the program crash if filename does not exist
10     for line in inf:
11         csv_str=line.strip()
12         (id,dy,s1,i1,s2,i2,s3,i3)=csv_str.split(',')
13         plankting_data.append( { "set": id, "day":dy,
14             "shall1":s1,"is1":i1,"shall2":s2,"is2":i2,"shall3":s3,"is3":i3,})
15     return plankting_data
16
17 # (2) adapter to write plankting into a csvfile
18 def write_as_csv(filename,plankting_data):
19     print(f"writing into csv file <{filename}>")
20     ouf=open(filename,"w")
21     # keep it simple: let the program crash if filename does not exist
22     for ds in plankting_data:
23         ouf.write(f"{ds["set"]},{ds["day"]},{ds["shall1"]},{ds["is1"]},"
24             f"{ds["shall2"]},{ds["is2"]},{ds["shall3"]},{ds["is3"]}\n")
25     return ouf.close()

```

LF11c::EL:Konverter:xyz→json:Step-05:VP-0.2b

```

1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # function for converting a csv file of plankting data into a json file: VP-1.0
3
4  # (3) adapter to write plankting into a json file
5  def write_as_json(filename,planking_data):
6      print(f"writing into json file <{filename}>")
7      ouf=open(filename,"w")
8      # keep it simple: let the program crash if filename does not exist
9      ouf.write("\n")
10     successor=False
11     for ds in planking_data:
12         if successor: ouf.write(",\n\t{\n")
13         else:
14             ouf.write("\t{\n")
15             successor=True
16         ouf.write(f"\t\t\t\t\t\"set\": {ds[\"set\"]},\n\t\t\t\t\t\"day\": {ds[\"day\"]},\n"
17             f"\t\t\t\t\t\"shall1\" : {ds[\"shall1\"]},\n\t\t\t\t\t\"is1\" : {ds[\"is1\"]},\n"
18             f"\t\t\t\t\t\"shall2\" : {ds[\"shall2\"]},\n\t\t\t\t\t\"is2\" : {ds[\"is2\"]},\n"
19             f"\t\t\t\t\t\"shall3\" : {ds[\"shall3\"]},\n\t\t\t\t\t\"is3\" : {ds[\"is3\"]}")
20     )
21     ouf.write(" }")
22     ouf.write("\n\n")
23     return ouf.close()

```

LF11c::EL:Konverter:xyz→json:Step-05:VP-1.0

```

1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # for converting a csv file of planking data into a json file: VP-1.0
3  import sys
4  import mod_dsp_teta_conv_csv as csv_adapters
5  import mod_dsp_teta_conv_json as json_adapters
6
7  # intermediate representation and hard-coded testdata
8  td_hard_coded=[
9      { "set": 1, "day": "2026-01-01",
10        "shall1": 30, "is1": 32, "shall2": 20, "is2": 18, "shall3": 15, "is3": 15},
11      { "set": 2, "day": "2026-01-02",
12        "shall1": 30, "is1": 35, "shall2": 25, "is2": 22, "shall3": 20, "is3": 18} ]
13
14  planking_data_infile="dsp.lf/dsp.teta-c.csv"
15  pd_out_format="csv"
16
17  if ((len(sys.argv)>1)) :
18      match sys.argv[1]:
19          case "JSON" | "json": pd_out_format="json"
20          case "test": planking_data_infile="dsp.lf/dsp.teta-t.csv"
21  if ((len(sys.argv)>2)) :
22      match sys.argv[2]:
23          case "JSON" | "json": pd_out_format="json"
24
25  # main
26  planking_data=csv_adapters.fill_from_csv(planking_data_infile)
27  #planking_data=td_hard_coded
28  if (planking_data):
29      match pd_out_format:
30          case "json": json_adapters.write_as_json("res.lf/planking_data.json",planking_data)
31          case _: csv_adapters.write_as_csv("res.lf/planking_data.csv",planking_data)

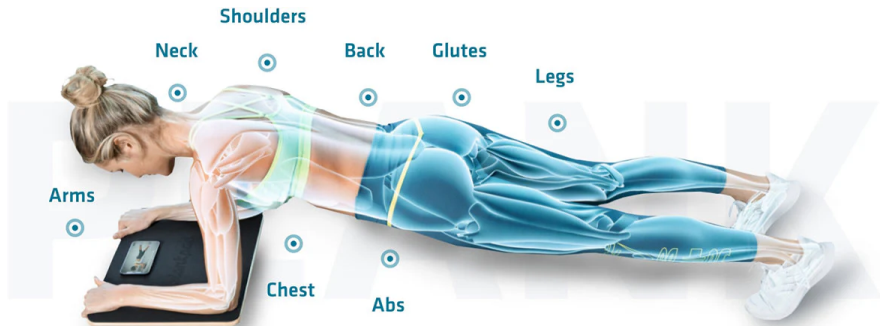
```

LF11c::EL:Konverter:xyz→json:Step-06:VP-1.1

```

1  # (C) 2026 K.Reincke: proTirone snippet [CC-BY-4.0]
2  # functions for reading and converting a csv file of plankting data: VP-1.1
3
4
5  # (4) adapter to write plankting data into a condensed csv file
6
7  # Idea: encode shall- and is-values into the same 2 byte integer:
8
9  def combine(shall_value, is_value) :
10     return (shall_value << 8) + is_value
11
12  def write_as_condensed_csv(filename,plankting_data):
13     print(f"writing into condensed csv file <{filename}>")
14     ouf=open(filename,"w")
15     # keep it simple: let the program crash if filename does not exist
16     for ds in plankting_data:
17         shis_1=combine(ds["shall1"],ds["is1"])
18         shis_2=combine(ds["shall2"],ds["is2"])
19         shis_3=combine(ds["shall3"],ds["is3"])
20         ouf.write(f"{ds["set"]},{ds["day"]},{shis_1},{shis_2},{shis_3}\n")
21     return ouf.close()
22
23
24

```



Data taken from a tool for strengthen your body: <https://plankpad.com/de>

Herzlichen Dank für Ihre Aufmerksamkeit!



Open Educational Resources

*Dieses Dokument stammt aus dem OER-Projekt <https://github.com/protirone/>, initiiert v. **Karsten Reincke, Hohenahr**. Es stellt **freie Unterrichtsmaterialien** samt **Quellcode** unter den Bedingungen der **CC-BY-4.0-Lizenz** zur Verfügung: Sie werden so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

Bildnachweise

-  von Wikimedia Commons. **Lizenziert** unter **CC0**. Bereitgestellt auf **Wikimedia**.
-  von Karsten Reincke. **Lizenziert** unter **proTirone-Logo-License**. Bereitgestellt auf **github**. (may only be used as logo for proTirone)
-  von plankpad.com. **Lizenziert** unter **proprietärer Lizenz**. Bereitgestellt auf **Plankpad.com**. (permission to use requested)